COPY Command: Demo

To Begin with the Lab

Summary of the Lab

In this lab, data was loaded into an existing **Amazon Redshift** cluster from an external text file instead of letting Redshift generate the sample data. A separate schema was created to isolate the imported data, and a sales table was defined with the required columns and data types. The source file was uploaded to **Amazon S3**, and an **IAM** role with S3 access was created and associated with the Redshift cluster so the COPY command could read the file securely. After the file was copied into the new sales table, the load was validated using a row-count query and a sample sales query. The results confirmed that the file-based data load worked correctly and that the loaded data matched the expected dataset.

Understanding Loading Data with COPY in Amazon Redshift

Amazon Redshift uses the COPY command to load large datasets efficiently from **Amazon S3** into tables. This is useful because S3 acts as a staging location for files such as TXT or CSV data before the records are imported into Redshift. The IAM role gives Redshift permission to read the S3 file without storing long-term credentials in the query. After loading, the data can be verified with SQL queries just like any native Redshift table.

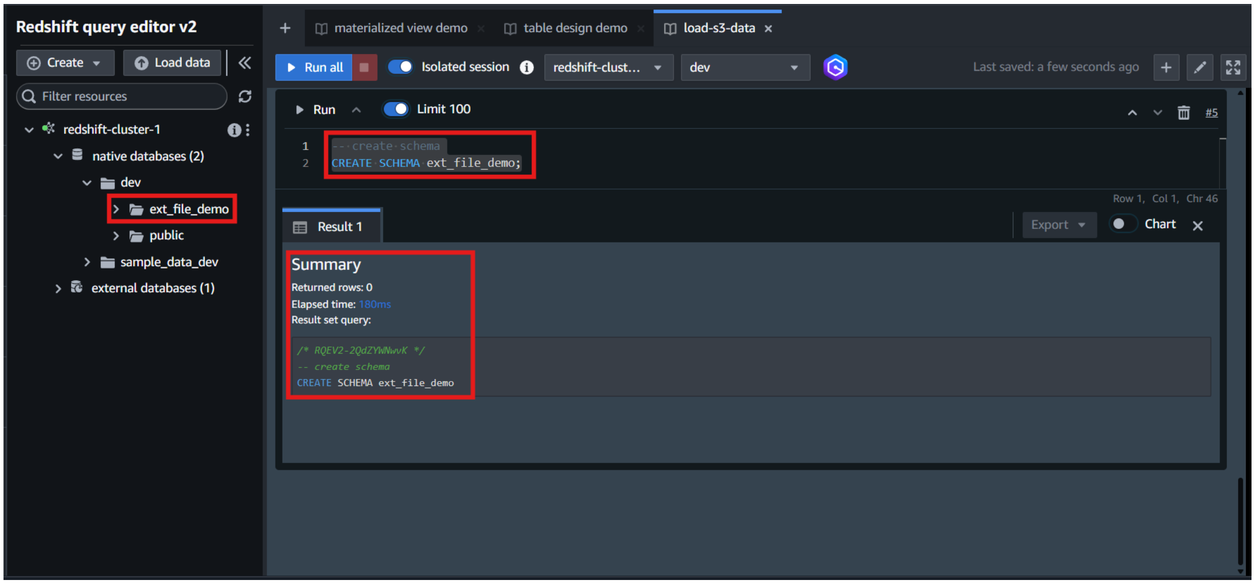
Example:

In this lab, the sales_tab.txt file contained about 172,000 sales records separated by tabs. The file was uploaded to **Amazon S3**, the Redshift cluster was given access through an **IAM** role, and the data was loaded into the sales table inside a new schema. A count query then confirmed that roughly 172,000 rows were imported successfully, showing that the external file was loaded correctly.

Lab Steps

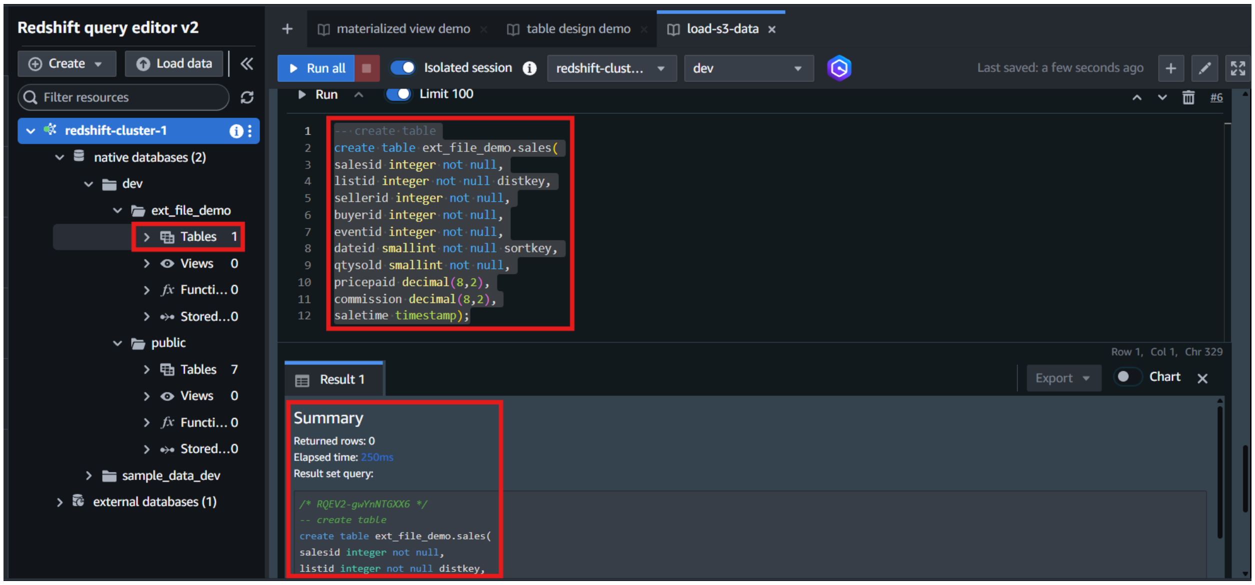
- Sign in to the AWS Management Console
- Open the existing **Amazon Redshift** cluster
- Open **Query Editor v2**
- Create a new schema to isolate the imported data

```
CREATE SCHEMA ext_file_demo;
```

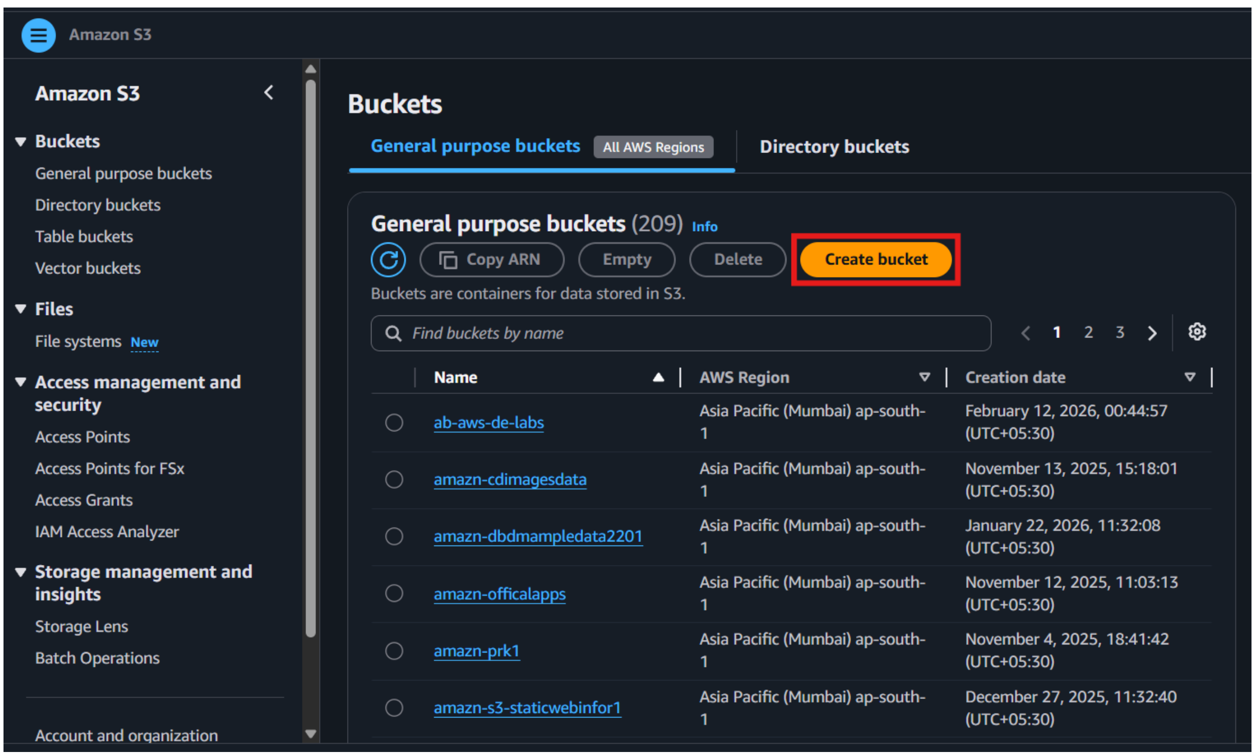


- Run the query
- Verify that the new schema appears in the cluster objects panel
- Create the sales table inside the new schema with the required columns and data types

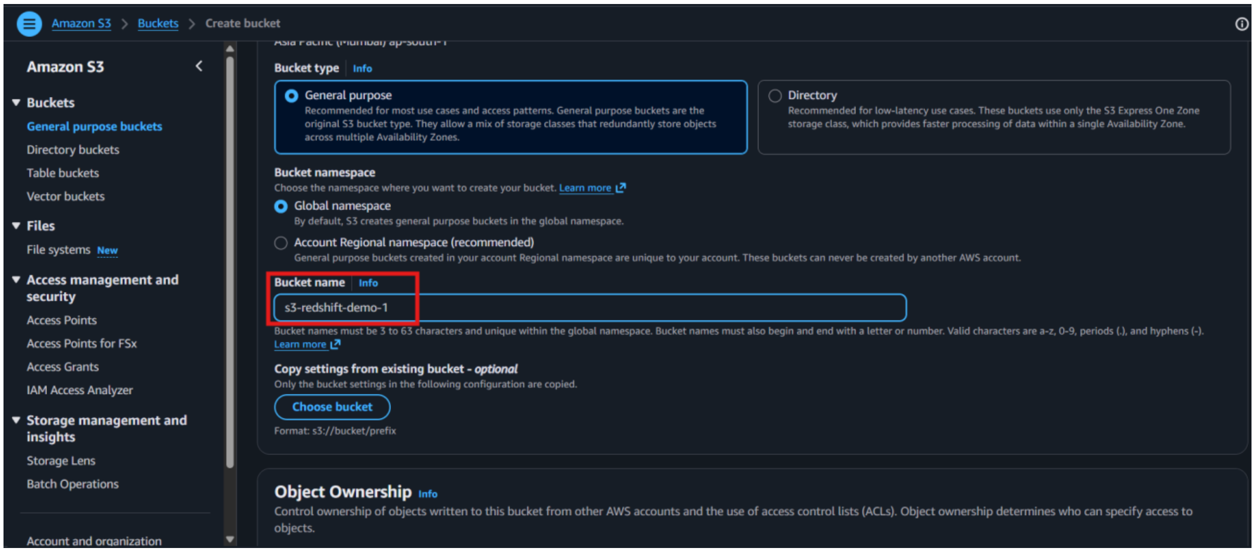
```
create table ext_file_demo.sales(
salesid integer not null,
listid integer not null distkey,
sellerid integer not null,
buyerid integer not null,
eventid integer not null,
dateid smallint not null sortkey,
qtysold smallint not null,
pricepaid decimal(8,2),
commission decimal(8,2),
saletime timestamp);
```



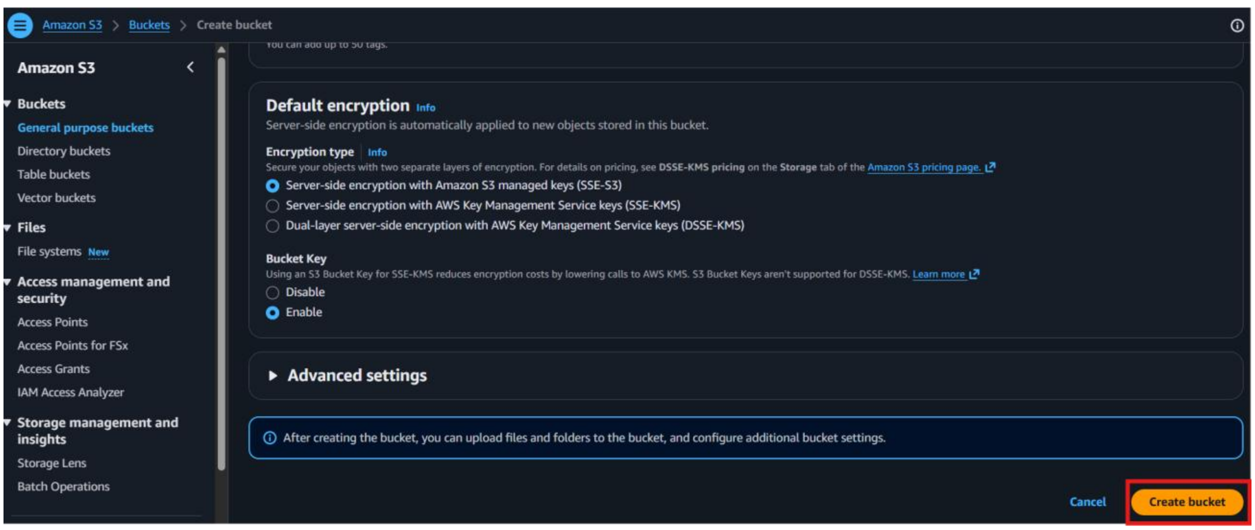
- Run the query
- Verify that the sales table is created under txt_file_demo
- Open **Amazon S3**
- Click **Create bucket**



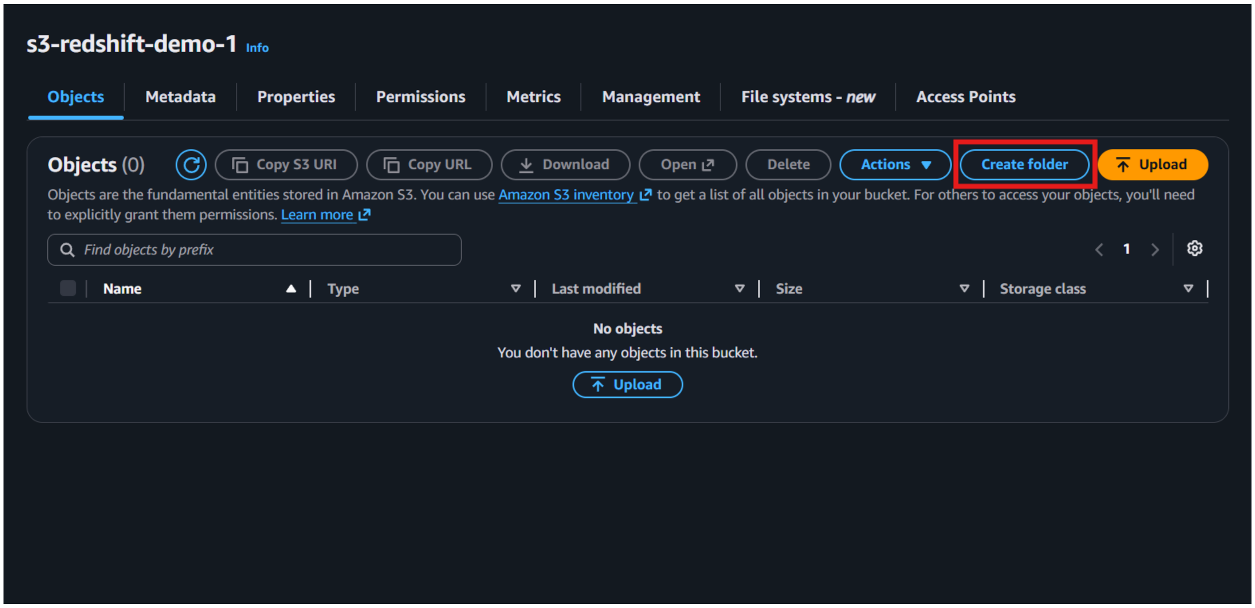
- Enter a unique bucket name



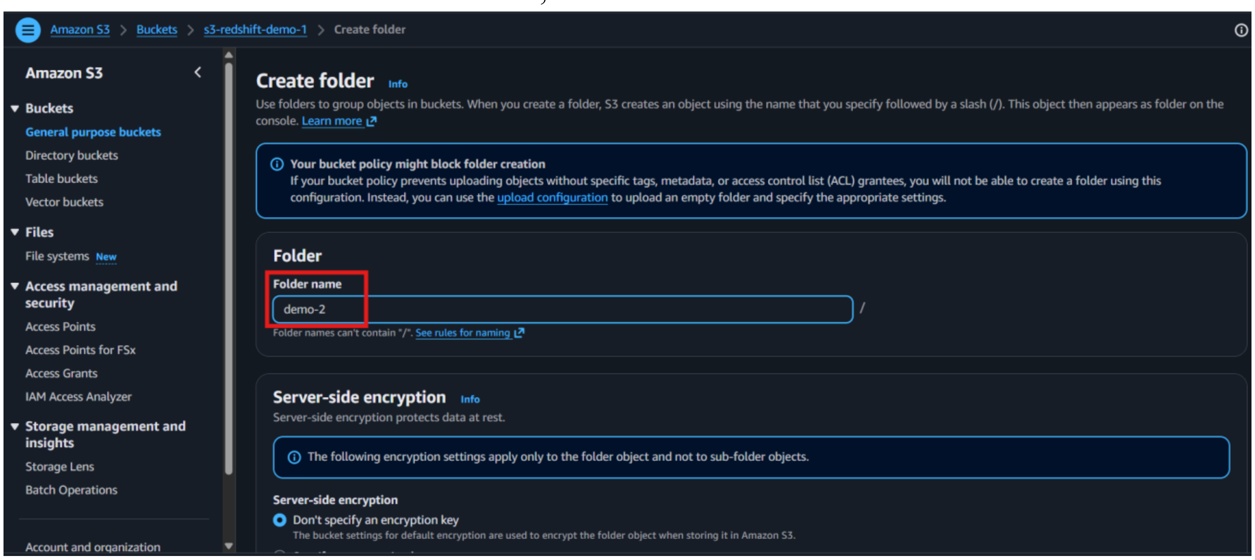
- Keep the default settings
- Click **Create bucket**



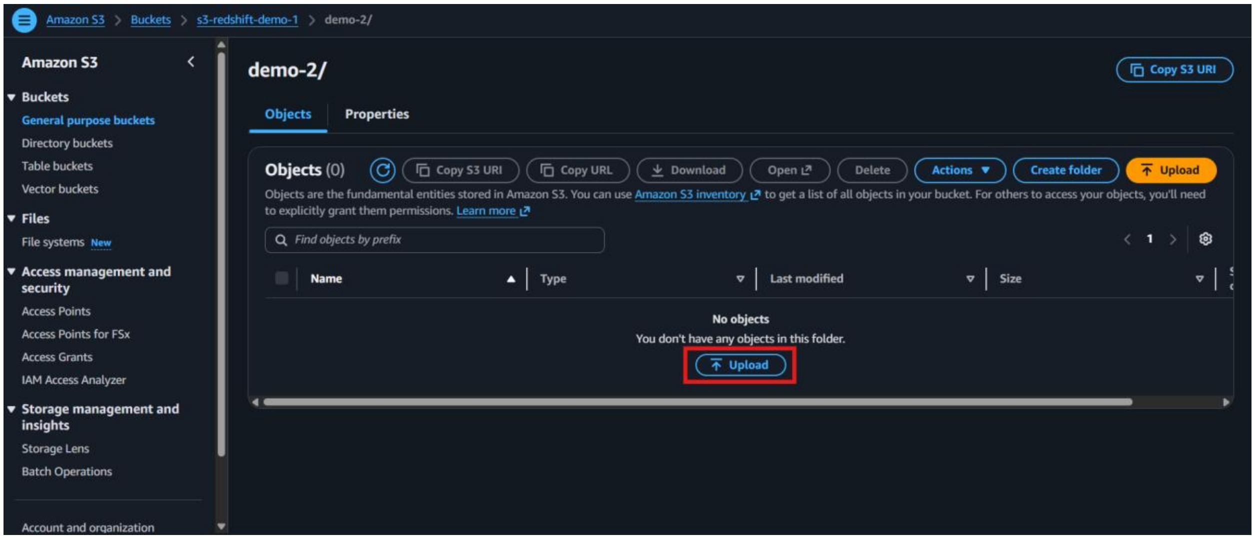
- Create a folder inside the bucket for organization



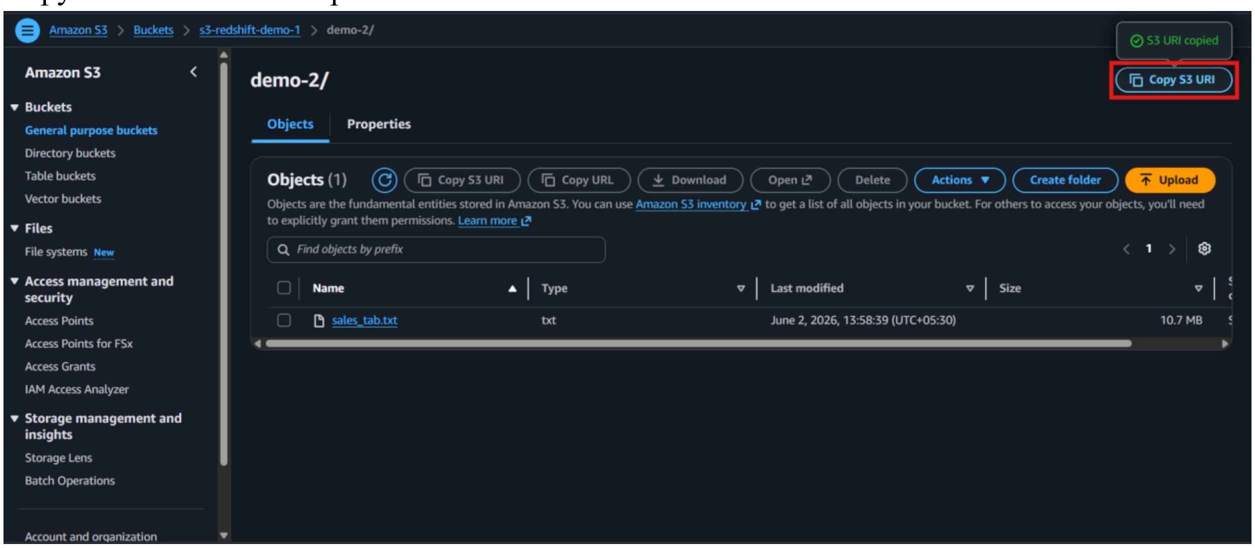
- Enter the name for folder and scroll down, click **Create Folder**



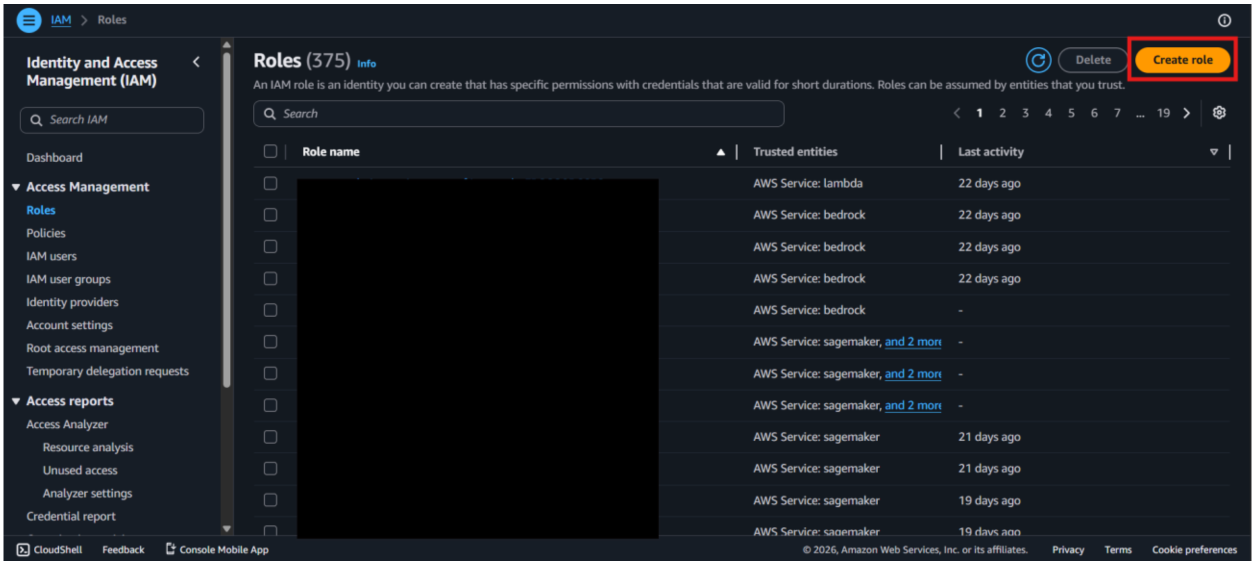
- Click **Upload** and choose file to upload



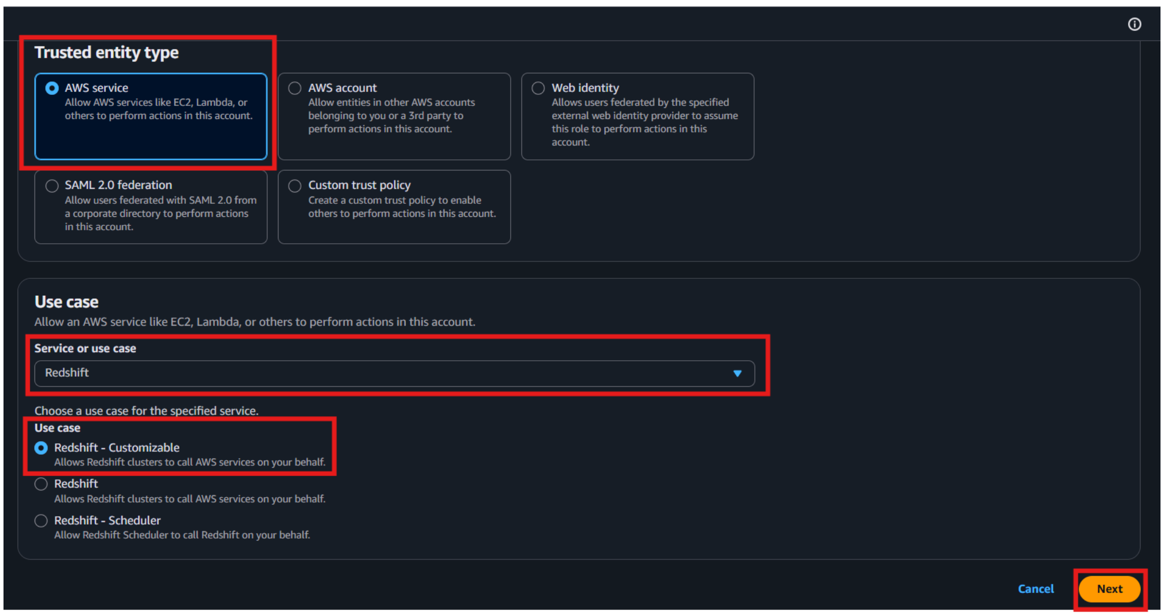
- Upload the sales_tab.txt file to the folder
- Copy the S3 URI of the uploaded file



- Open **IAM**
- Navigate to **Roles**
- Click **Create role**

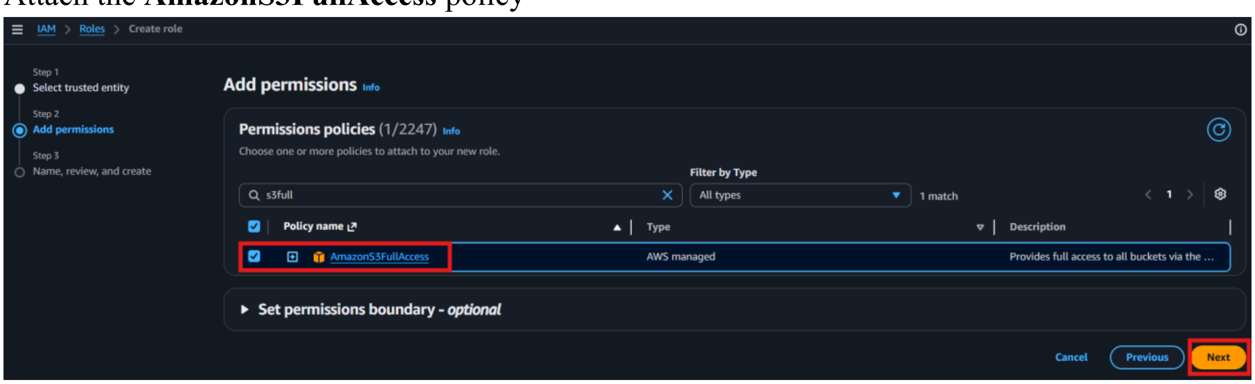


- Select **Amazon Redshift** as the AWS service

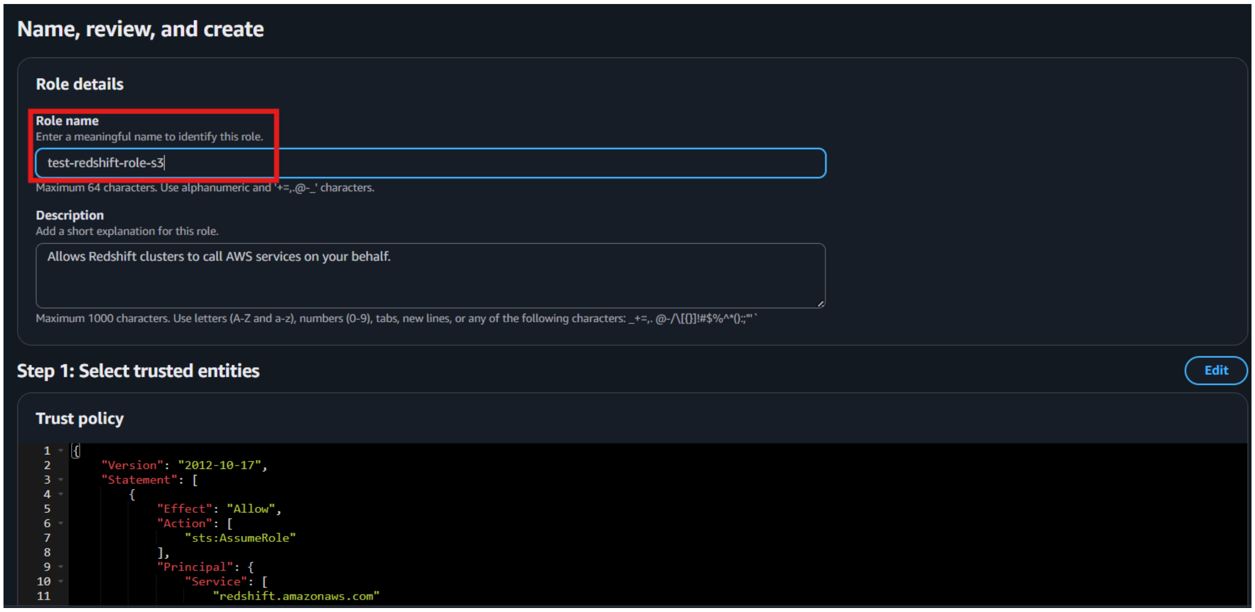


Add Required Permissions

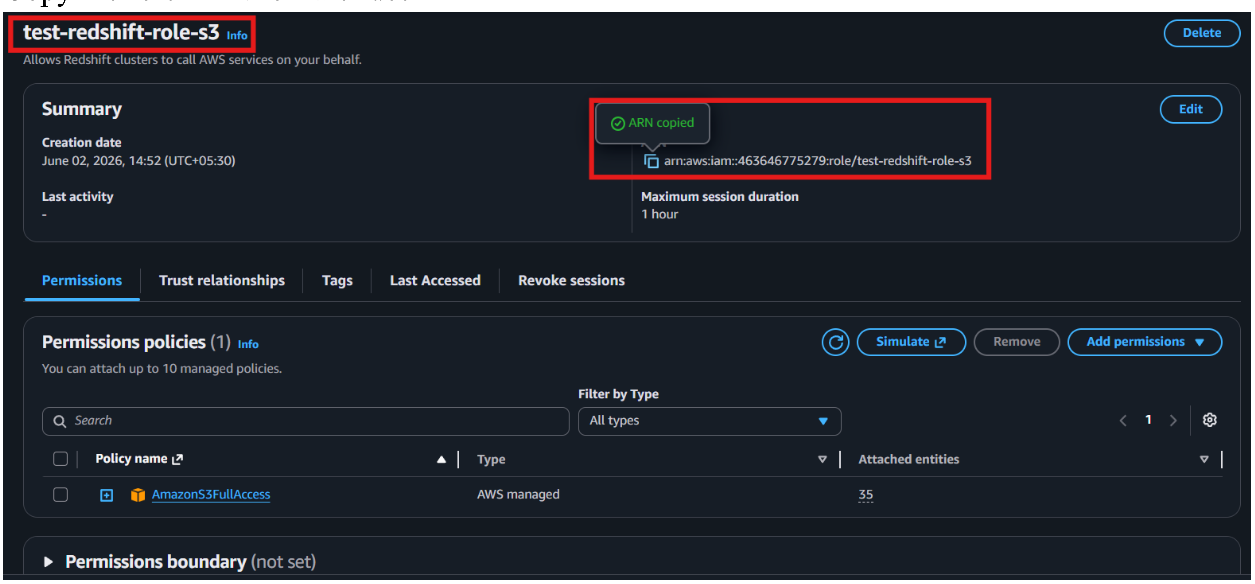
- Attach the **AmazonS3FullAccess** policy



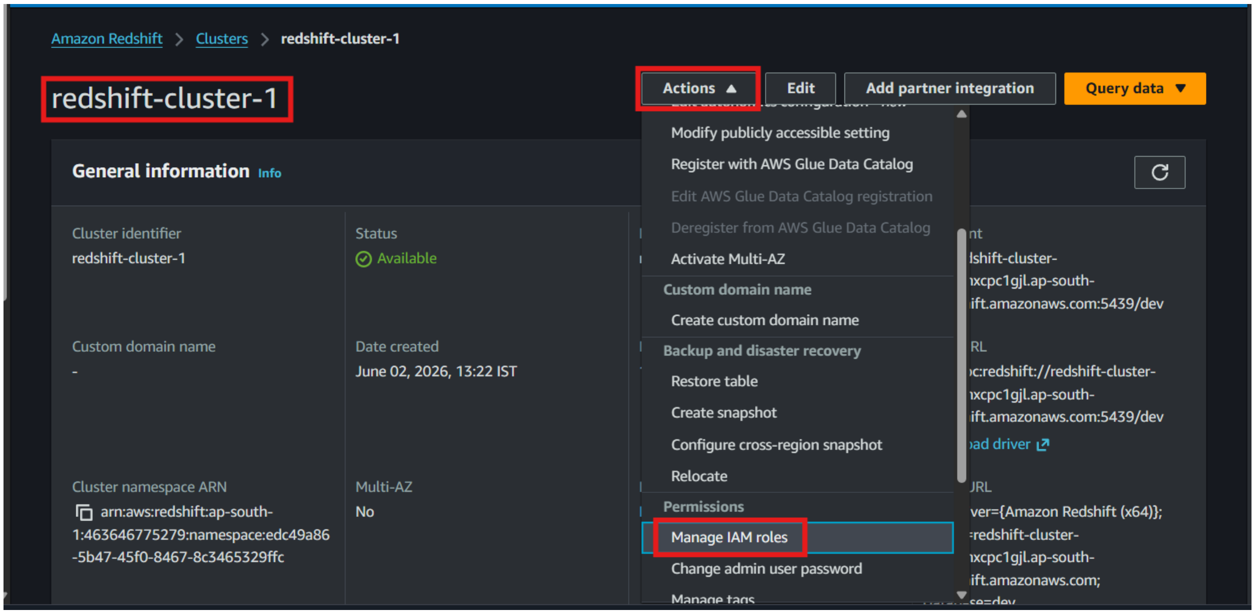
- Enter a role name for the Redshift S3 access role



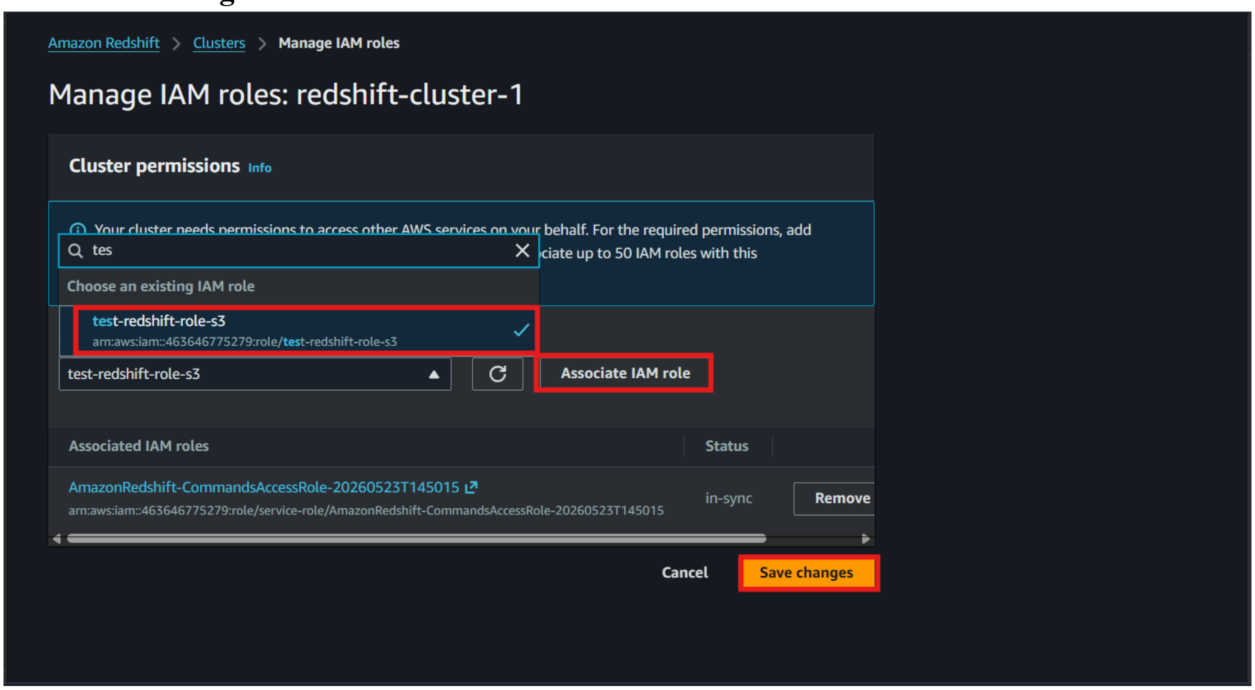
- Click **Create role**
- Copy the role ARN for later use



- Return to the Redshift cluster
- Click **Actions → Manage IAM roles**

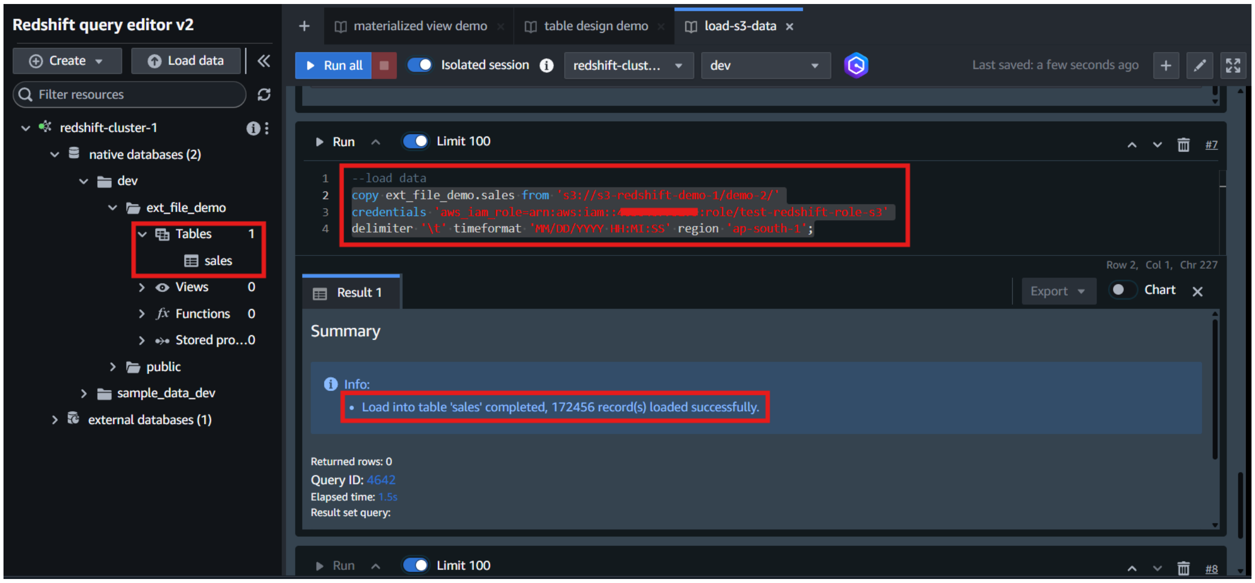


- Select the newly created IAM role
- Associate the role with the cluster
- Click **Save changes**



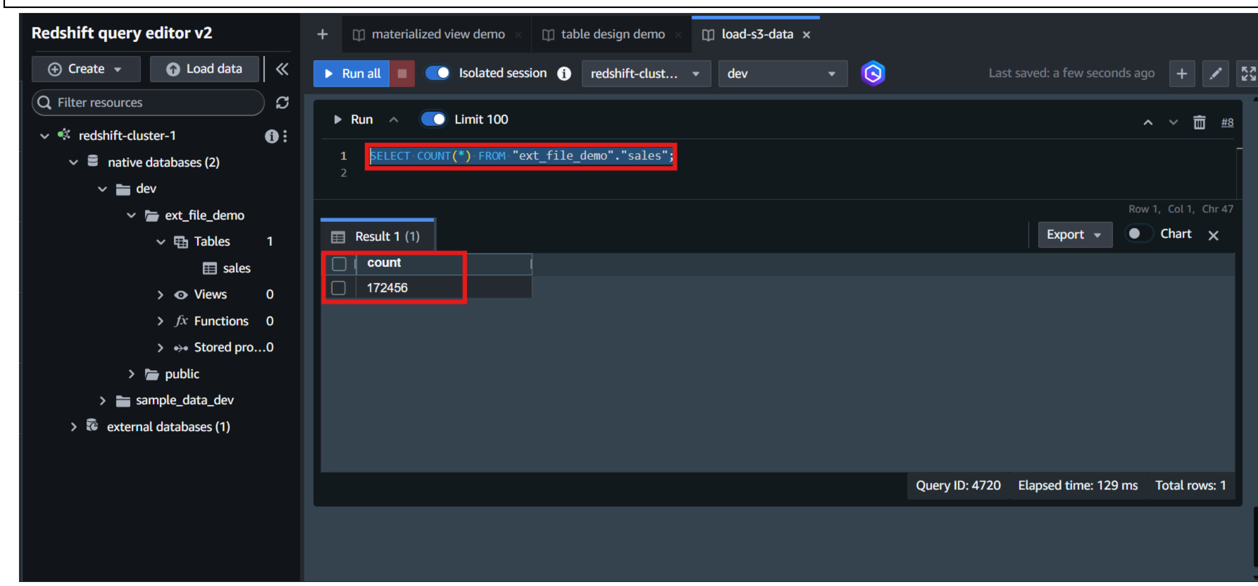
- Return to **Query Editor v2**
- Run the COPY command using the S3 file URI, role ARN, tab delimiter, time format, and AWS Region

```
copy ext_file_demo.sales from 's3://{s3_location}/'
credentials 'aws_iam_role=arn:aws:iam::$account_id:role/test-role-redshift-s3'
delimiter '\t' timeformat 'MM/DD/YYYY HH:MI:SS' region 'ap-south-1';
```



- Run the command
- Verify that the data loads successfully into the sales table
- Run a row count query to validate the import

```
SELECT COUNT(*)
FROM "ext_file_demo"."sales";
```



- Verify that the count is approximately 172,000 rows
- Run a sample query against the loaded table
- Verify that the query returns the expected sales results
- Confirm that the external file data was successfully loaded into **Amazon Redshift** using **Amazon S3** as the staging location and the COPY command as the loading method.